The third part of the `comment()` view is the *form renderer*. This is like the final part of most views, creating a context dictionary and calling the `render()` shortcut. Note that this code can be reached for two reasons: because the HTTP method was GET or because the method was POST and the form was not valid. In the latter case, the form object contains error information that can be displayed to the user.

Add the code in listing 7.2 to RiffMates/content/views.py. You also need to create RiffMates/content/urls.py with a `path` object for the view, similar to RiffMates/bands/urls.py. To register all the paths in content/urls.py, make sure to add an `include` line inside RiffMates/RiffMates/urls.py for the new file. Review chapter 4 if you need a refresher on using app-based URL routing files.

Listing 7.2 A view accepting both HTTP GET and POST to handle comment form data

```
# RiffMates/content/views.py
from django.core.mail import send_mail
from django.shortcuts import render, redirect

from content.forms import CommentForm

def comment(request):
    if request.method == 'GET':
        form = CommentForm()

    else: # POST
        form = CommentForm(request.POST)

        if form.is_valid():
            name = form.cleaned_data["name"]
            comment = form.cleaned_data["comment"]

            message = f"""\
                Received comment from {name}\n\n
                {comment}
            """

            send_mail("Received comment", message,
                "admin@example.com", ["admin@example.com"],
                fail_silently=False)
            return redirect("comment_accepted")

    # Was a GET, or Form was not valid
    data = {
        "form": form,
    }

    return render(request, "comment.html", data)
```

The CommentForm class with name and comment fields

In the GET handler portion of the code, create an empty form object.

The POST handler portion of the code, reached on form submit

Create a form object, populating it with the data from the submit.

Validate the form.

The cleaned_data dictionary contains the form contents.

Use send_mail() to send an email from the admin to the admin.

Turn fail_silently off.

Redirect the user to the comment_accepted view, using its path name.

The rendering part of the view, used in both the GET case and the POST case if the form wasn't valid

You're not quite done yet. The `comment()` view redirects to a view named `comment_accepted()`, which someone has to write. Luckily, you're someone! Add the code from the following listing to RiffMates/content/views.py. Don't forget to register this